

## Fase 1: Fundamentação e Ambiente de Desenvolvimento (Emulação)

A etapa inicial foca na compreensão do funcionamento do espaço de kernel (*Kernel Space*) e na configuração do ambiente de compilação isolado para a arquitetura alvo.

- **Estudo da API do Kernel Linux:** O desenvolvimento de um *Loadable Kernel Module* (LKM) exige familiaridade com as estruturas de dados e macros específicas do Kernel Linux, que diferem drasticamente da programação convencional em espaço de usuário (*User Space*).
- **Integração com a Toolchain:** Para compilar o módulo, será estritamente necessário utilizar a *Cross-Compiler Toolchain* (ex: `riscv64-unknown-linux-gnu-gcc`) gerada e exportada pela equipe responsável pela infraestrutura do Sistema Operacional.
- **Validação Básica via QEMU:** O primeiro marco prático desta fase é escrever, compilar e carregar um driver "Hello World" básico no sistema Linux genérico que estará rodando no emulador QEMU. Isso validará o processo de carga (`insmod`) e descarga (`rmmod`) de módulos na arquitetura RISC-V.

## Fase 2: Estruturação do Driver de Caractere e Mapeamento de Memória

Nesta fase, o driver começa a tomar sua forma definitiva, estabelecendo as interfaces tanto com o espaço de usuário quanto com os endereços físicos do hardware.

- **Criação do Dispositivo de Caractere:** O driver deve ser registrado no sistema operacional como um dispositivo de caractere (*Character Device*). O objetivo é expor um nó de acesso no sistema de arquivos, especificamente o arquivo `/dev/npu_ternaria`, por onde a aplicação de IA fará a comunicação.
- **Implementação das *File Operations*:** É necessário estruturar a *struct file\_operations*, mapeando as chamadas de sistema (syscalls) provenientes da aplicação para as rotinas internas do driver, definindo o comportamento fundamental das funções `.open`, `.release`, `.read`, `.write` e `.unlocked_ioctl`.
- **Memory-Mapped I/O (MMIO):** O kernel não consegue acessar endereços físicos diretamente. O desenvolvedor deverá receber o mapa de memória do hardware contendo o Endereço Base da NPU e seus respectivos *offsets*. A partir do momento em que a NPU estiver descrita no arquivo *Device Tree Source* (DTS), o driver utilizará a função `ioremap()` (ou equivalentes da API gerenciada, como `devm_ioremap`) para converter os endereços físicos do hardware em memória virtual acessível pelo kernel.

+2

## Fase 3: Lógica de Controle, Transferência e Sincronização

Esta é a fase crítica do desenvolvimento, onde a lógica de comunicação ponta a ponta é implementada para acionar efetivamente o acelerador *multiplierless*.

- **Transferência Segura de Dados:** Como o driver faz a ponte entre a aplicação e o hardware, ele não pode acessar ponteiros de usuário diretamente por questões de segurança e paginação. Deve-se implementar o uso das macros `copy_from_user()` e `copy_to_user()`.

- **Injeção de Dados na NPU:** Uma vez que os buffers contendo as matrizes do *dataset* e os pesos empacotados em variáveis de 32 bits cheguem ao kernel (via chamadas `write()` ou `ioctl()`), o driver deverá escrevê-los ordenadamente nos Endereços Físicos correspondentes aos Registradores de Dados de Entrada da NPU.  
+2
- **Controle e Sincronização de Hardware:**
  - \* **Start:** Após enviar os dados, o driver deve escrever um sinal no Registrador de Controle para iniciar a operação da NPU.
  - **Polling/IRQ:** O driver deve implementar uma rotina de *polling* (verificação contínua do registrador) ou configurar uma interrupção de hardware para monitorar o Registrador de Status, aguardando o sinal *Done/Ready* indicando que a multiplicação terminou.
- **Retorno dos Resultados:** Ao confirmar a finalização, o driver deve ler os dados processados do Registrador de Dados de Saída e enviá-los de volta à aplicação do espaço de usuário através da resposta à chamada `read()`.  
+1

#### Fase 4: Integração Física, Depuração e Otimização

A etapa final ocorre quando o hardware é sintetizado no FPGA e o Linux embarcado é inicializado fisicamente na placa.

- **Deploy no SoC Físico:** O módulo final (`.ko`) deverá ser carregado no sistema operacional rodando no hardware físico, testando a ausência de *Kernel Panics* ou falhas de paginação (*Page Faults*).
- **Testes de Integração Fim a Fim:** Realizar baterias de teste executando o código de inferência em linguagem C, validando se o caminho Aplicação → Driver → Hardware → Driver → Aplicação flui com resultados matematicamente corretos.
- **Otimização para *Benchmarking*:** Para subsidiar a etapa crítica de coleta de métricas, o desenvolvedor do driver deve garantir que o *overhead* (custo de transição de contexto) gerado pelas chamadas de sistema e pelo próprio driver seja minimizado, assegurando que o tempo de execução medido reflita de forma justa e limpa o ganho de velocidade proporcionado pelo hardware acelerador.